

PatchTST

Long-Term

Time Series Forecasting

PRESENTED BY

Ayaan Khan

CS584 - Spring 2025 - Illinois Tech

Problem Statement

Long-term time series forecasting faces two compounding challenges.

CHALLENGE 1

O(L²) attention cost

Quadratic complexity limits how far back the model can look. Long look-back windows — often the most useful in time series — become prohibitively expensive.

CHALLENGE 2

Point-wise tokens

Individual timesteps carry no semantic meaning — unlike words. Real patterns live at the subsequence level, invisible to point-wise attention.

THE AWKWARD CONTEXT

Zeng et al. (2023) showed that **DLinear** — a simple decomposition + linear projection — beats Informer, Autoformer, and FEDformer on these benchmarks.

So the real question: can a Transformer actually win here — and if so, why?

Background on Transformers

Why vanilla Transformers struggle with time series.

SELF-ATTENTION LIMITATION

Point-wise attention misses local structure

Standard Transformers attend over individual timesteps. But a single timestamp has no semantic content the way a word in a sentence does.

The patterns that actually drive forecasting accuracy — trends, cycles, shocks — live at the subsequence level. Point-wise attention is blind to them.

CROSS-VARIATE MIXING

Informer, FEDformer & Autoformer overfit

Earlier Transformers embed all variables at each timestep into a single token. The model is then forced to learn correlations across channels that may not exist.

On small datasets like ETT (~17K points), this produces classic spurious-correlation overfitting.

PatchTST's Solution

Two simple ideas that fix both problems at once.

INNOVATION 1

Patching

Group consecutive timesteps into patches — ViT for time series.

336 → 41

tokens, with patch_len=16, stride=8

Complexity: $O(L^2) \rightarrow O((L/S)^2)$
≈ 67× reduction in attention cost.

INNOVATION 2

Channel Independence

All 7 channels share a backbone, but never attend to each other.

✓ Kills spurious cross-variate overfitting.

✓ Shared weights generalize across all C variables.

✓ Effectively multiplies training samples by C.

Architecture

Input → RevIN → patches → Transformer encoder → flat head → forecast.



The encoder itself is a standard pre-norm Transformer. The novelty lives in steps 2 and 3 — how the input is normalized and tokenized.

Implementation Details

Built from scratch in PyTorch. No HuggingFace, no author code.

HYPERPARAMETERS

d_model	128
n_heads	8
n_layers	3
d_ffn	512
dropout	0.2
patch_len	16
stride	8
batch size	128
lr	1e-4 → 1e-6 (cosine)
early stop	patience = 10

HARDWARE

~8 sec / epoch

Apple M4 Pro (24GB) · PyTorch MPS backend · 7.5x faster than CPU

MPS WORKAROUNDS (PyTorch 2.8)

1. `unfold()` returns non-contiguous views → crashes `nn.Linear` on MPS. Fix: `.contiguous()`.
2. `nn.TransformerEncoderLayer` MPS crash on certain dims. Fix: custom MHSA using `F.scaled_dot_product_attention`.

RevIN Normalization

Reversible Instance Normalization — the single most impactful component.

THE PROBLEM

Distribution shift

ETT data is non-stationary. Mean and variance drift over the 17,420 timesteps.

Train: months 1–12

Val: months 13–16

Test: months 17–20

Each window sees a different distribution. Model trained on one struggles on another.

THE FIX

Per-window normalize

1. Compute μ, σ per window (cache them)
2. Normalize input to zero mean / unit var
3. Learnable γ, β for recoverable scale
4. Forecast in normalized space
5. Denormalize output: $\hat{y} \times \sigma + \mu$

Datasets & Training Protocol

Standard ETT benchmark. Standard splits. Standard horizons.

17,420

HOURLY TIMESTEPS
~2 years of data

7

CHANNELS
6 load vars + air temp

4

HORIZONS
96, 192, 336, 720 steps

ETTh1

Electricity Transformer Temperature. Relatively stationary — train and test distributions align.

ETTh2

Same sensors, different transformer. Real distributional shift between train and test periods.

SPLIT 12 mo train · 4 mo val · 4 mo test METRICS MSE, MAE in z-score space (matches paper protocol)

Main Results

Normalized MSE on ETTh1 and ETTh2. Lower is better. Winners per row in coral.

Dataset	Horizon	PatchTST	PatchTST + RevIN	DLinear	Winner
ETTh1	96	0.4609	0.4469	0.4457	DLinear
ETTh1	192	0.5087	0.4996	0.4908	DLinear
ETTh1	336	0.5472	0.5497	0.5440	DLinear
ETTh1	720	0.6525	0.6729	0.6393	DLinear
ETTh2	96	0.2574	0.2467	0.2374	DLinear
ETTh2	192	0.3452	0.2990	0.3042	PatchTST+RevIN
ETTh2	336	0.4212	0.3518	0.3776	PatchTST+RevIN
ETTh2	720	0.6183	0.4578	0.5957	PatchTST+RevIN

Key takeaway: PatchTST+RevIN wins 3 of 4 ETTh2 horizons — 23% better than DLinear at h=720. Without RevIN, DLinear wins all 8 combinations.

The Key Finding

Why PatchTST+RevIN wins on ETTh2 but not ETTh1.

ETTh1 · STATIONARY

DLinear wins all 4 horizons

Train and test distributions align closely. RevIN has nothing to correct — in fact, it adds noise at long horizons because the denorm step applies look-back stats that don't match the forecast period.

When data is near-linear, a linear model genuinely is the right answer.

ETTh2 · DISTRIBUTION SHIFT

PatchTST+RevIN wins 3 of 4

Oil temperature and load patterns evolve across the time span. RevIN re-anchors each window at inference, regardless of absolute level.

At h=720, that translates to a 23% MSE improvement over DLinear — and the gap grows with horizon.

Ablation: Patch Length & Stride

ETTh1, horizon = 96. Patch length essentially tied; stride=16 is the real winner.

Patch Length

Patch Len	n_patches	MSE	MAE
8	42	0.4507	0.4481
16 (default)	41	0.4522	0.4500
32	39	0.4512	0.4495

Stride

Stride	n_patches	MSE	MAE
4	81	0.4520	0.4480
8 (default)	41	0.4571	0.4551
16 (best)	21	0.4464	0.4495

Patch length: All three values within 0.3% of each other — PatchTST is robust to patch length choice in this regime.

Stride: Stride=16 (non-overlapping) clearly wins — fewer redundant tokens, better attention diversity.

Bonus: Halving stride quadruples attention cost — less overlap is a double win (accuracy AND compute).

Ablation: Look-back Window

ETTh1, horizon = 96. Paper's default seq_len=336 wins. Performance degrades sharply at 512.

seq_len	n_patches	MSE	MAE
96	11	0.4557	0.4510
192	23	0.4546	0.4530
336 (default, best)	41	0.4522	0.4490
512	63	0.4708	0.4655

What this means

Monotonic improvement: 96 → 192 → 336 each improves in turn (paper default wins).

Sharp reversal at 512: +4.1% MSE vs 336 — longer is not always better.

Why it breaks down: The flat head's parameter count scales with n_patches. At seq_len=512 it has ~6x more params than at 96 — overfits on 8,640 training samples.

Gap vs. Published Results

ETTh1, normalized MSE. Both my models lag the paper by similar margins — points to training duration, not architecture.

Horizon	Mine (PatchTST+RevIN)	Paper	Gap	My DLinear	Paper DLinear
96	0.4469	0.370	+21%	0.4457	0.386
192	0.4996	0.413	+21%	0.4908	0.437
336	0.5497	0.422	+30%	0.5440	0.481
720	0.6729	0.447	+51%	0.6393	0.456

Not a bug

DLinear shows a similar gap. A PatchTST-only issue would leave DLinear alone.

Training duration

Early stopping (patience=10) terminates at epoch 20–40. Paper trains longer with full cosine schedule.

Relative rankings hold

Model ordering matches the paper — which is what the architectural comparison actually tests.

Conclusions

Five takeaways from implementing PatchTST from scratch.

- 01

RevIN is the most impactful component

23% gain at ETTh2 h=720. Marginal on stationary ETTh1.
- 02

PatchTST+RevIN validates the paper on ETTh2

Wins 3 of 4 horizons. Transformer + right normalization beats DLinear on harder data.
- 03

Without RevIN, DLinear wins everywhere

Zero wins across 8 combinations. Transformer capacity alone is not enough.
- 04

Larger stride reduces error (and compute)

Non-overlapping patches win on both axes — fewer redundant tokens.
- 05

Look-back gains saturate beyond 336

Monotonic improvement up to paper default, then sharp overfitting at seq_len=512.

References

PatchTST

Nie, Nguyen, Sinthong & Kalagnanam. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. ICLR 2023.

DLinear

Zeng, Chen, Zhang & Xu. Are Transformers Effective for Time Series Forecasting? AAAI 2023.

RevIN

Kim et al. Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift. ICLR 2022.

ViT

Dosovitskiy et al. An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale. ICLR 2021.

Informer

Zhou et al. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. AAAI 2021.

FEDformer

Zhou et al. FEDformer: Frequency Enhanced Decomposed Transformer for Long-term Series Forecasting. ICML 2022.

Thank you — questions welcome.